

User Manual

for S32K1_S32M24X AE Driver

Document Number: UM2AEASRR21-11 Rev0000R3.0.0 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	3
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	7
3.5 Driver Limitations	7
3.6 Driver usage and configuration tips	7
3.6.1 Note when configuring the AE module	7
3.6.2 How to use config to use NMI handler	9
3.7 Runtime errors	11
3.8 Symbolic Names Disclaimer	11
3.9 Usage in non AUTOSAR context	11
4 Configuration Parameters	14
4.1 Ae	15
4.1.1 Module Ae	16
4.1.2 Container AeGeneral	17
4.1.3 Parameter AeVersionInfoApi	17
4.1.4 Parameter AeDevErrorDetect	17
4.1.5 Parameter AeEnableUserModeSupport	18
4.1.6 Parameter AeMultipartitionSupport	18
4.1.7 Reference AeEcucPartitionRef	19
4.1.8 Container AeGeneralVendorSpecific	19
4.1.9 Parameter AeTimeoutMethod	19
4.1.10 Parameter AeTimeoutDuration	20
4.1.11 Container AeHvmVmConfig	20
4.1.12 Parameter AeHvmVoltageMonitorInputSelect	21
4.1.13 Parameter AeVmHighVoltageDetectInterruptEn	21
4.1.14 Parameter AeVmLowVoltageDetectInterruptEn	22
4.1.15 Parameter AeHvmHighVoltageDetectEn	22
4.1.16 Parameter AeHvmLowVoltageDetectEn	22
4.1.17 Parameter AeHvmLowDetectRefVoltage	23
4.1.18 Parameter AeHvmHighDetectRefVoltage	23
4.1.19 Container AeHvmHviConfig	24
4.1.20 Container AeHvmHviGeneralConfig	24
4.1.21 Parameter AeHvmHviAnalogchannel	24
4.1.22 Parameter AeHvmAnalogInput	25
4.1.23 Parameter AeHvmUnderControlIsrCallback	25
4.1.24 Container AeHvmHviInstanceConfig	26
4.1.25 Parameter AeHvmHviChannelIdx	26
4.1.26 Parameter AeHVIIInputDigitalEn	26
4.1.27 Parameter AeHVIIInterruptInputDigitalEn	27
4.1.28 Parameter AeHviDigitalInputPolarity	27
4.1.29 Parameter AeHviDividerSelect	28
4.1.30 Parameter AeHVIPullUpEn	28

4.1.31	Parameter AeHVIPullDownEn	29
4.1.32	Container AeSpi	29
4.1.33	Container AeSpiSequence	29
4.1.34	Parameter AeSpiAccessSynchronous	30
4.1.35	Reference AeSpiSequenceName	30
4.1.36	Container AeIrqConfig	30
4.1.37	Parameter ClearEventsFaultsFlagEn	31
4.1.38	Container AeIntConfig	31
4.1.39	Parameter AeIntName	31
4.1.40	Parameter AeIsrEnabled	32
4.1.41	Parameter AeIsrPriority	32
4.1.42	Parameter AeIntHandler	33
4.1.43	Container AeGHSConfig	33
4.1.44	Parameter AeGHSProcessingCallout	33
4.1.45	Parameter AeGHSHandlerEnable	34
4.1.46	Container AeGHSHandlerConfig	34
4.1.47	Parameter AeGHSName	35
4.1.48	Parameter AeHandlerEnabled	35
4.1.49	Parameter AeGHSPriority	35
4.1.50	Parameter AeGHSHandler	36
4.1.51	Container AeNMIConfig	36
4.1.52	Parameter NMIInterruptEnable	37
4.1.53	Reference AePlatformSourceName	37
4.1.54	Container CommonPublishedInformation	37
4.1.55	Parameter ModuleId	38
4.1.56	Parameter VendorId	38
4.1.57	Parameter VendorApiInfix	38
4.1.58	Parameter ArReleaseMajorVersion	39
4.1.59	Parameter ArReleaseMinorVersion	39
4.1.60	Parameter ArReleaseRevisionVersion	40
4.1.61	Parameter SwMajorVersion	40
4.1.62	Parameter SwMinorVersion	41
4.1.63	Parameter SwPatchVersion	41
4.2	AEC	42
4.2.1	Module Aec	42
4.3	HVM	43
4.3.1	Module Hvm	44
5	Module Index	45
5.1	Software Specification	45
6	Module Documentation	46
6.1	AEC Driver	46
6.1.1	Detailed Description	46
6.1.1.1	Functions Expectation called for read/write value from/to AE's registers in polling/interrupt mode:	46
6.1.2	Data Structure Documentation	47
6.1.2.1	struct Aec_Ip_SpiConfigType	47
6.1.2.2	struct Aec_Ip_IntRouteConfigType	47
6.1.2.3	struct Aec_Ip_GHSRouteConfigType	48
6.1.3	Types Reference	48
6.1.3.1	Aec_Ip_IntHandlerType	48
6.1.3.2	Aec_Ip_GHSHandlerType	48
6.1.4	Enum Reference	48

6.1.4.1 Aec_Ip_StatusType	48
6.1.5 Function Reference	50
6.1.5.1 Aec_Ip_Init()	50
6.1.5.2 Aec_Ip_SpiWrite()	50
6.1.5.3 Aec_Ip_SpiRead()	50
6.1.5.4 Aec_Ip_Spi_Frame_Counter()	51
6.1.5.5 Aec_Ip_DecodeGlobalHealthStatus()	51
6.1.5.6 Aec_Ip_SetRaw()	51
6.2 HVM Driver	52
6.2.1 Detailed Description	52
6.2.1.1 Functions Expectation called first for configuration initilized:	52
6.2.2 Data Structure Documentation	53
6.2.2.1 struct Ae_HviHighVoltageInputConfig	53
6.2.2.2 struct Ae_VmConfig	53
6.2.2.3 struct Ae_HviChannelConfig	53
6.2.3 Enum Reference	53
6.2.3.1 Hvm_Ip_StatusType	53
6.2.3.2 Hvm_Ip_OutputStatusType	53
6.2.3.3 Hvm_Ip_ChannelType	54
6.2.3.4 Hvm_Ip_HighVoltDetectRefType	54
6.2.3.5 Hvm_Ip_LowVoltDetectRefType	54
6.2.3.6 Hvm_Ip_VoltageMonitorInputType	54
6.2.3.7 Hvm_Ip_DigitalInputPolarityType	54
6.2.3.8 Ae_AnalogInputType	55
6.2.3.9 Hvm_Ip_DivVoltageType	55
6.2.4 Function Reference	55
6.2.4.1 Hvm_Ip_Init()	55
6.2.4.2 Hvm_Ip_AdcMeasureConfig()	55
6.2.4.3 Hvm_Ip_DigitalInputEventConfig()	56
6.2.4.4 Hvm_Ip_VoltageMonitorConfig()	56
6.2.4.5 Hvm_Ip_ReadFaultsEventsStatus()	56
6.2.4.6 Hvm_Ip_FaultsEventsProcess()	57
6.3 Aeernet Driver	58
6.3.1 Detailed Description	58
6.3.2 Data Structure Documentation	60
6.3.2.1 struct Ae_Ipw_ConfigType	60
6.3.3 Macro Definition Documentation	60
6.3.3.1 AE_SID_INIT	60
6.3.3.2 AE_SID_WRITEREGISTER	60
6.3.3.3 AE_SID_READREGISTER	60
6.3.3.4 AE_SID_GETVERSIONINFO	60
6.3.3.5 AE_SID_ADCMEASURECONFIG	60
6.3.3.6 AE_SID_DIGITALINPUTCONFIG	61
6.3.3.7 AE_SID_VOLTAGEMONITORCONFIG	61
6.3.3.8 AE_SID_DECODEGLOBALHEALTHSTATUS	61
6.3.3.9 AE_SID_FRAMECOUNTER	61
6.3.3.10 AE_SID_FAULTSEVENTSPROCESS	61
6.3.3.11 AE_SID_READFAULTSEVENTSSTATUS	61
6.3.3.12 AE_SID_SETRAW	62
6.3.3.13 AE_E_INV_CTRL_IDX	62
6.3.3.14 AE_E_UNINIT	62
6.3.3.15 AE_E_PARAM_POINTER	62
6.3.3.16 AE_E_INV_PARAM	62
6.3.3.17 AE_E_INV_MODE	62

6.3.3.18	AE_E_INIT_FAILED	63
6.3.3.19	AE_E_ALREADY_INITIALIZED	63
6.3.3.20	AE_E_PARAM_CONFIG	63
6.3.4	Enum Reference	63
6.3.4.1	Ae_StateType	63
6.3.5	Function Reference	63
6.3.5.1	Ae_Init()	63
6.3.5.2	Ae_ReadRegister()	63
6.3.5.3	Ae_WriteRegister()	64
6.3.5.4	Ae_DecodeGlobalHealthStatus()	64
6.3.5.5	Ae_Frame_Counter()	64
6.3.5.6	Ae_AdcMeasureConfig()	65
6.3.5.7	Ae_DigitalInputEventConfig()	65
6.3.5.8	Ae_VoltageMonitorConfig()	65
6.3.5.9	Ae_FaultsEventsProcess()	65
6.3.5.10	Ae_ReadFaultsEventsStatus()	65
6.3.5.11	Ae_SetRaw()	66
6.3.5.12	Ae_GetVersionInfo()	66



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	07.03.2025	NXP RTD Team	S32K1_S32M24X Real-Time Drivers AUTOSAR R21-11 Version 3.0.0

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductors' AE Driver for S32M24x.

AE Driver configuration parameters description can be found in the Tresos Configuration Plugin section.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32m241_lqfp64
- s32m242_lqfp64
- s32m243_lqfp64
- s32m244_lqfp64

All of the above microcontroller devices are collectively named as S32K1_S32M24X. Note: MWCT part numbers contain NXP confidential IP for Qi Wireless Power

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.

- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
DEM	Diagnostic Event Manager
DET	Default Error Tracer
AE	Application Extension
AEC	Application Extension Controller
HVM	High Voltage Module
VM	Voltage Monitor
HVI	High Voltage Input
SPI	Serial Peripheral Interface
MCU	Micro controller Unit
N/A	Not Available
RAM	Random Access Memory

- The term "Ae Driver" is related to the software handling the Application Extension.
- The term "Application" is used for the software utilizing the AE Driver.

2.5 Reference List

#	Title	Version
1	S32K1xx Reference Manual	Rev.14.1, 01/2024
2	S32M24x Reference Manual	Rev. 5, 12/2024
3	S32K1xx Data Sheet	Rev. 14, 08/2021
4	S32M2xx Data Sheet	Rev. 7, 12/2024
5	S32K116 Errata	Mask Set Errata for Mask 0N96V Rev. 17/JAN/2024, 1/2024
6	S32K118 Errata	Mask Set Errata for Mask 0N97V Rev. 15/JAN/2024, 1/2024
7	S32K142 Errata	Mask Set Errata for Mask 0N33V Rev. 15/JAN/2024, 1/2024
8	S32K144 Errata	Mask Set Errata for Mask 0N57U Rev. 15/JAN/2024, 1/2024
9	S32K144W Errata	Mask Set Errata for Mask 0P64A Rev. 03/JUL/2024, 7/2024
10	S32K146 Errata	Mask Set Errata for Mask 0N73V Rev. 15/JAN/2024, 1/2024
11	S32K148 Errata	Mask Set Errata for Mask 0N20V Rev. 15/JAN/2024, 1/2024
12	S32M242 Errata	Mask Set Errata for Mask N33V+P69K Rev.1, 8/2024
13	S32M244 Errata	Mask Set Errata for Mask P64A+P69K Rev.1, 8/2024

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)
- [Usage in non AUTOSAR context](#)

3.1 Requirements

None.

3.2 Driver Design Summary

The AE Driver controls the Application Extension Controller (AEC) module and High Voltage Monitor(HVM) of the S32M24x device. It provides the following features:

For AEC:

- A SPI interface for microcontroller communication to read/ write to AE's registers for polling and interrupt mode.
- Read Frame Counter status.
- Allow to control Raw bit.
- Decode global health status of frame.
- Read status of faults and events through interrupt_out raised.

For HVM:

- A voltage divider to divide down different input voltage levels to the system voltage.
- Event generation, when the input logic level changes. This works also when the clock is off, so you can use this digital input interrupt for example to wake up the device from sleep mode.

- Event observation through interrupt generation, reading of an interrupt flag in register, or polling the event status from a register.
- Analog output, e.g. for reading the input voltage with an on-chip ADC.
- Selectable pull-up and pull-down connections on the input for open input detection.
- Fixed input voltage division by 11 in the connected HVI when a voltage monitor is enabled.
- Event generation, when the input voltage is out of customer-defined limits. You can configure the upper and lower limits of the allowed voltage range of the input signal.
- Observation of any voltage out of defined limits event through interrupt generation, reading of an interrupt flag in a register, or polling the event status from a register.

3.3 Hardware Resources

The SPI interface communication to microcontroller pin mapping can be done using the file **S32M24x_IOMU**↔**X.xlsx** attached to the Reference Manual.

3.4 Deviations from Requirements

None.

3.5 Driver Limitations

None.

3.6 Driver usage and configuration tips

3.6.1 Note when configuring the AE module

- The AE module must be configured according to the steps below. Follow steps below to config that:
 1. Config clock for SPI in range 4Mhz to 10Mhz

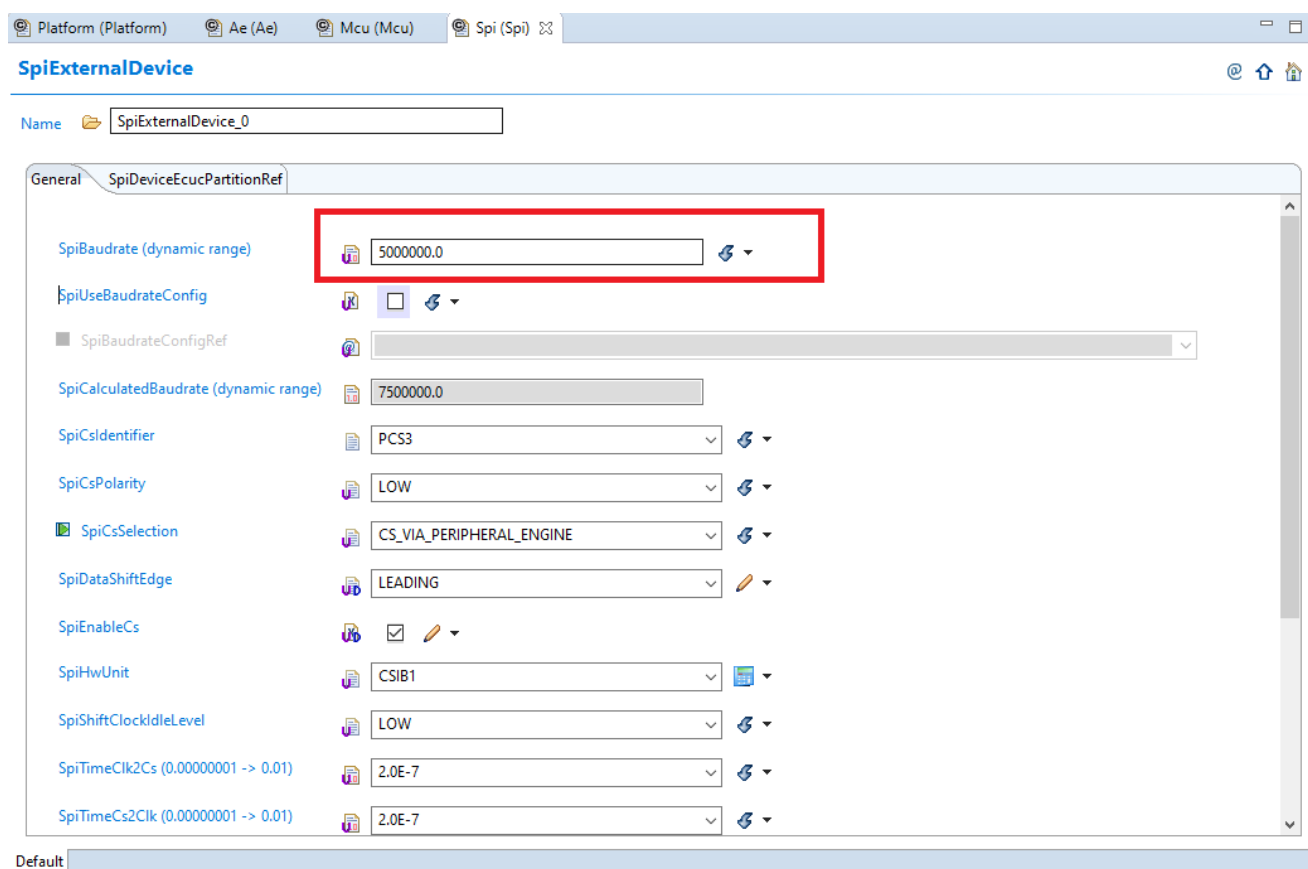


Figure 3.1 Config clock SPI

This parameter is the communication baudrate. This parameter allows using a range of values, from the point of view of configuration tools, from Hz up to Mhz. This field is used only in MASTER mode. This field is used only for LPSPI The baudrate must be in allowed range

LPSPI in normal Master mode:

- RUN MODE: [1 to 10Mhz]
- HSRUN MODE: [1 to 14Mhz] only support for K14x
- VLPR MODE: [1 to 2Mhz] for K1xx and [1 to 500Khz] for K14xW

FLEXIO in Master:

Note: Due to synchronization delays, the setup time for the serial input data is 1.5 FlexIO clock cycles, so the maximum baud rate is divide by 4 of the FlexIO clock frequency.

Theoretical baud rate =

- $$\text{FLEXIO_CLK (BUS_CLK)} \div (16 \times 2 \times (\text{TIMCMPn[COMP]} + 1)) \text{ FLEXIO_CLK (BUS_CLK)} \div (2 \times (\text{TIMCMPn[COMP]} + 1)) -$$
- RUN MODE: [1 to $(\text{BUS_CLK}/4) \div (2 \times (\text{TIMCMPn[COMP]} + 1)) = 6\text{Mhz}$]
 - HSRUN MODE: [1 to $(\text{BUS_CLK}/4) \div (2 \times (\text{TIMCMPn[COMP]} + 1)) = 7\text{Mhz}$] only support for K14x
 - VLPR MODE: [1 to $(\text{BUS_CLK}/4) \div (2 \times (\text{TIMCMPn[COMP]} + 1)) = 0.5\text{Mhz}$] for K11x, K14x and [1 to $(\text{BUS_CLK}/4) \div (2 \times (\text{TIMCMPn[COMP]} + 1)) = 62.5\text{KhzMhz}$] for K14xW

Note

The precision of this value depends SPI clock source configuration. If the driver cannot generate correct of the value, approximate value will be used.

Figure 3.2 Range of LPSPI support on S32M24x

2. Increase timeout duration

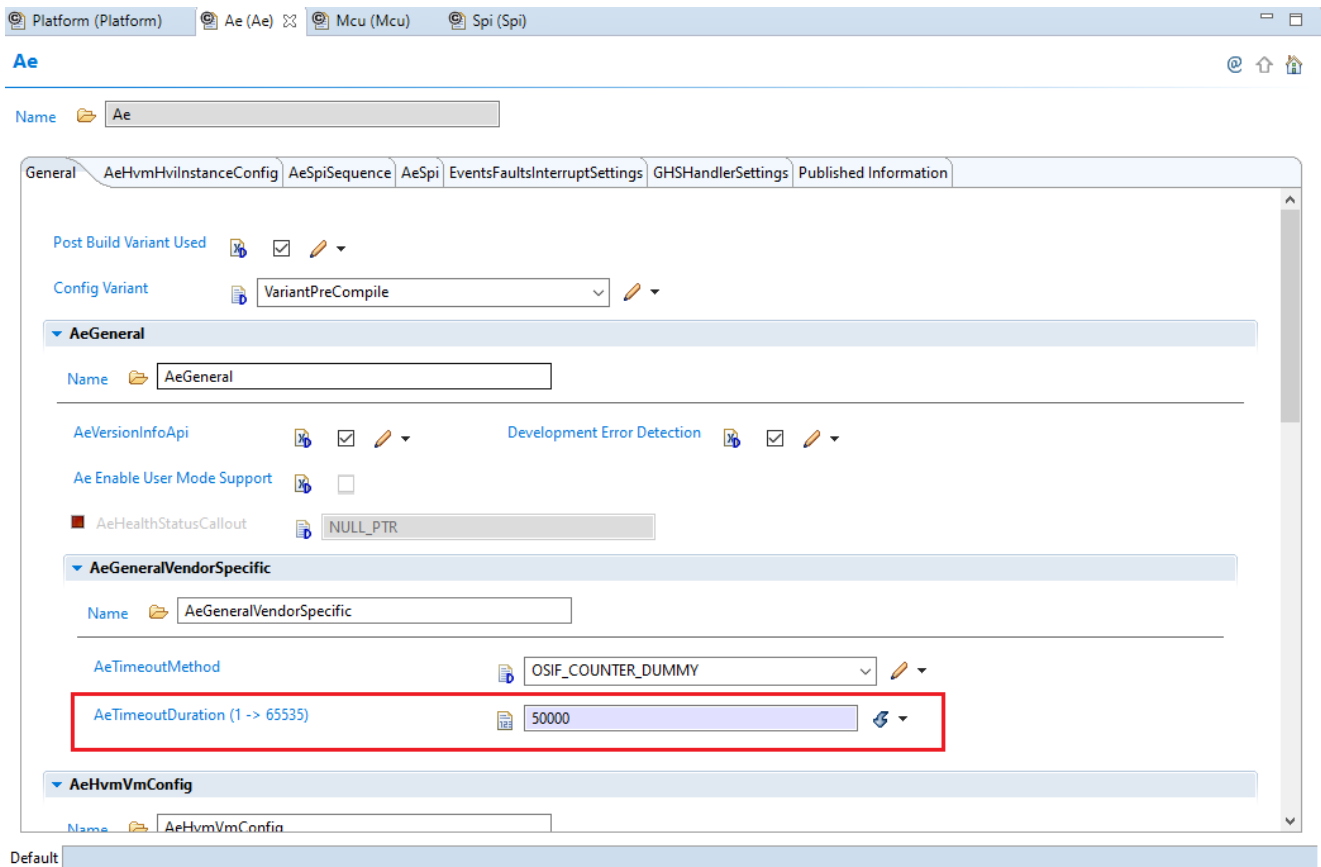


Figure 3.3 Ae timeout duration

3.6.2 How to use config to use NMI handler

1. Make sure NMI is enable.
2. Enable node NMIIInterruptEnable and select source platform

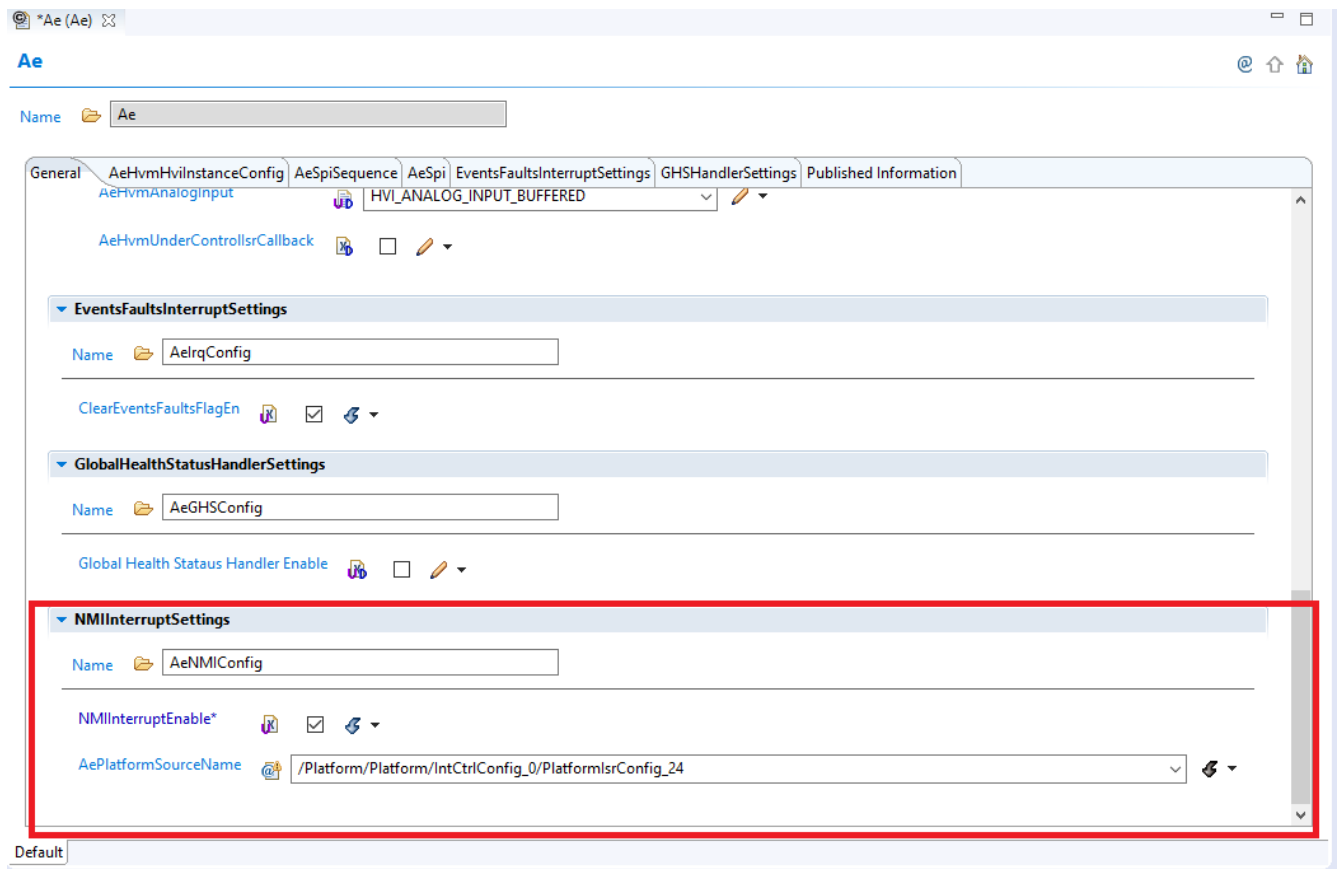


Figure 3.4 Config NMI and choose source interrupt

3. Application will set up the ISR and register it on the platform.

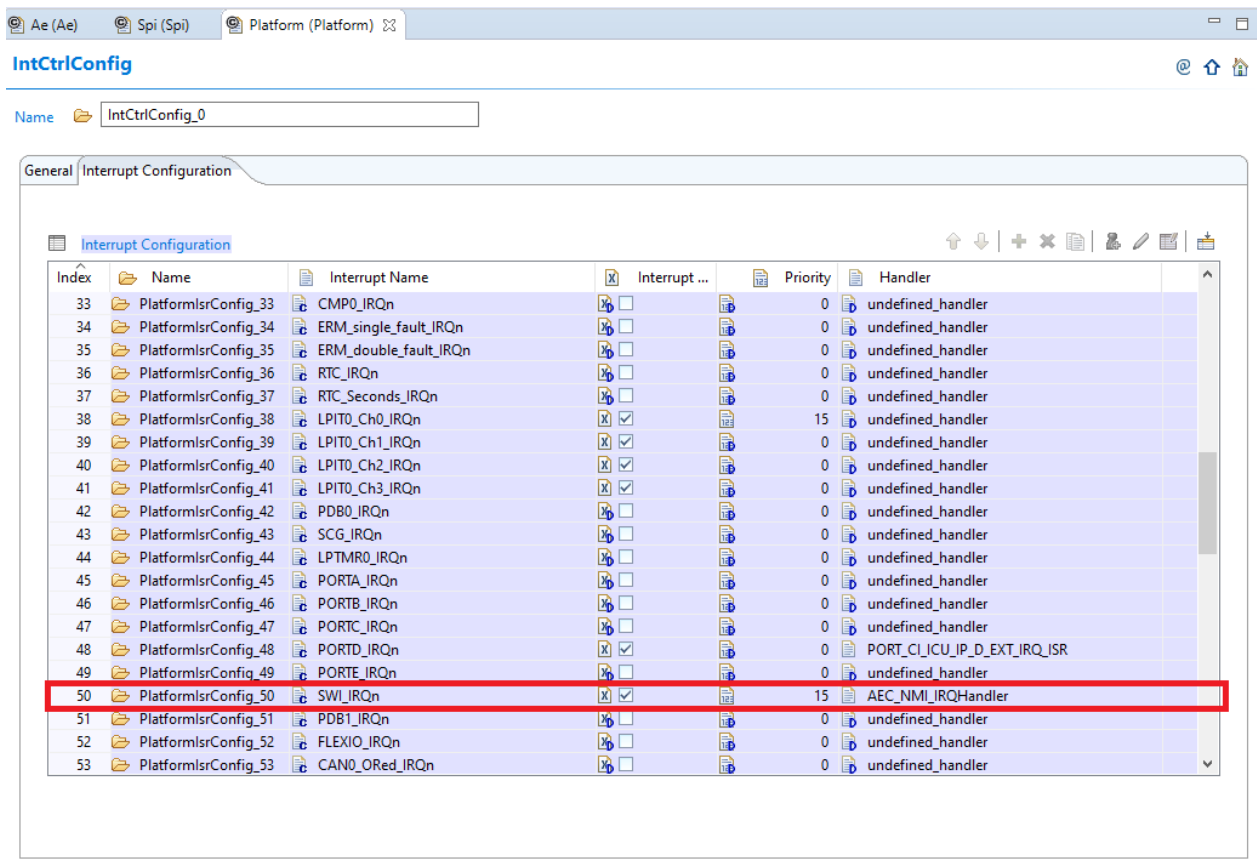


Figure 3.5 Config interrupt

4. In NMI handler, users needs to call function `Aec_Ip_NMISetupEventFaultHandler()` to disable NMI and trigger a interrupt that user register in platform before
5. In the ISR users needs to call the `Aec_Ip_InterruptSetupEventFaultHandler()` function to enable NMI and clear the interrupt flag

3.7 Runtime errors

None.

3.8 Symbolic Names Disclaimer

All containers having `symbolicNameValue` set to `TRUE` in the AUTOSAR schema will generate defines like: `#define <Mip>Conf_<Container_ShortName>_<Container_ID>`

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

3.9 Usage in non AUTOSAR context

For the non-ASR environments, both High-Level and IP layer interfaces are exposed and can be used by software units above. These may consist of either higher level NXP SW deliverables, like stacks and middleware, or the application code itself.

The goal of the product is to be versatile, easy to use and integrate with other SW environments, offering both standardized and optimized low-level interfaces.

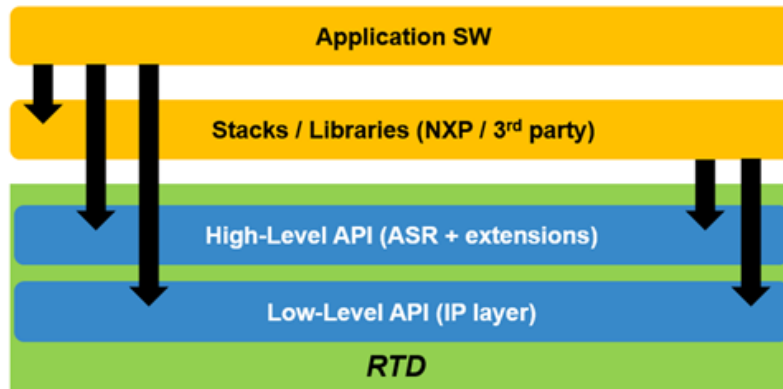


Figure 3.6 RTD Usage in non-ASR Context

In the non-AUTOSAR environment, there are no specific ECU states defined, but an external assumption for the application is that the correct sequence of initialization takes place prior to calling any runtime APIs.

The constraints considered for defining the APIs in the non-AUTOSAR environment are:

- Hardware resources used by the drivers must be properly initialized prior to runtime usage by calling the initialization function with configuration parameters generated by the configuration tools
- All dependencies must be initialized prior to the usage of a driver that depends on them; this translates to a general initialization sequence applicable for all applications, as follows:
 1. Initialization of the device clock tree so other periphery can be initialized
 2. Configuration of the pinout so peripherals with external connections can communicate with the outside world
 3. Configuration of the common resources (e.g. DMA channels, memory regions, access rights, etc) prior to the usage of any runtime functionality that has to benefit from this overall SoC configuration
 4. Initialization of specific drivers
 5. Runtime functionality
 6. Deinitialization (inverse initialization order)

High-level and IP layer functionalities cannot be used simultaneously over the same hardware resource. The granularity of the resource must be defined specifically for each driver, but as a general rule the default granularity considered is the peripheral instance (e.g. Gpt high level APIs cannot be called interchangeably with Pit_Ip low level APIs over the same PIT hardware instance).

The diagram below shows a generic, high-level sequence of calls to RTD interfaces in the non-AUTOSAR scenario:

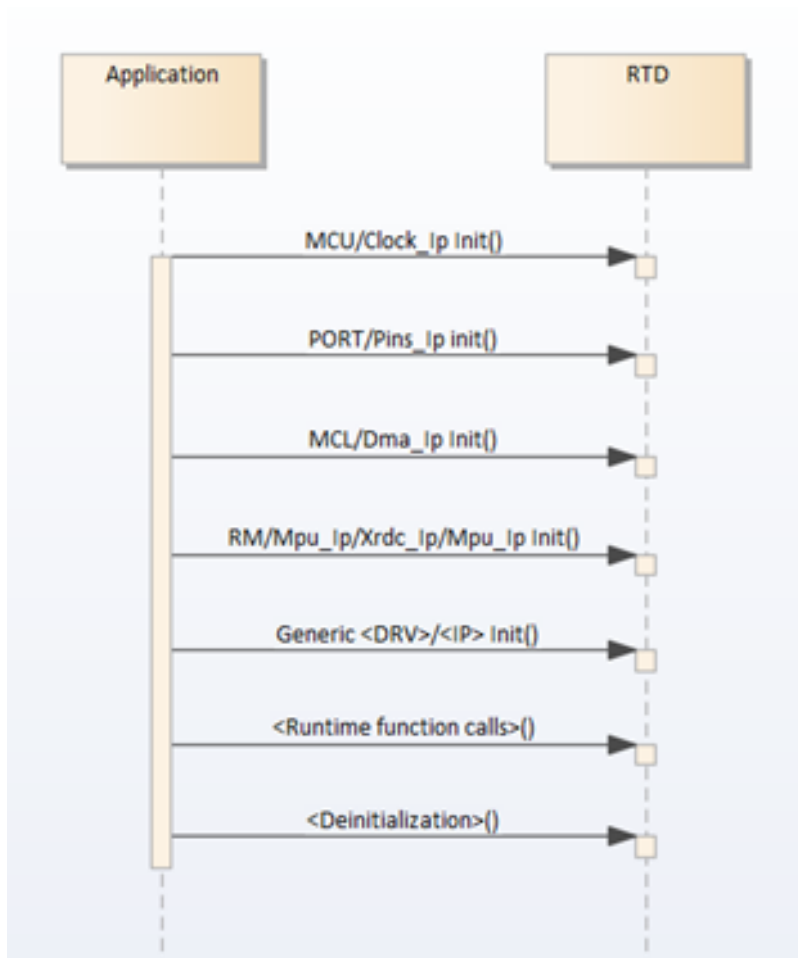


Figure 3.7 Non-AUTOSAR Sequence diagram

Chapter 4

Configuration Parameters

RTD provides configuration schema for the high level driver in EB Tresos proprietary format (**.xdm**), S32CT proprietary format (**.component**), but also in a standardized AUTOSAR format (**.epd**). For the low level driver configuration only the S32CT proprietary format (**.component**) is supported. The configuration data of the project can be saved in EB Tresos proprietary format (**.xdm**) and AUTOSAR standardized format (**.epc**) from EB Tresos and in S32CT proprietary format (**.mex**) and AUTOSAR standardized format (**.epc**) from S32 Design Studio. RTD provides templates for parsing the configuration data and generating configuration structures from it in XPath format, to be used with the EB Tresos generator and in JS format, to be used with the S32CT generator. Since the generation process is not standardized by AUTOSAR, only the two mentioned tools can be used with the provided templates. Using the same input configuration data, the same configuration structures will be obtained using any of the generators.

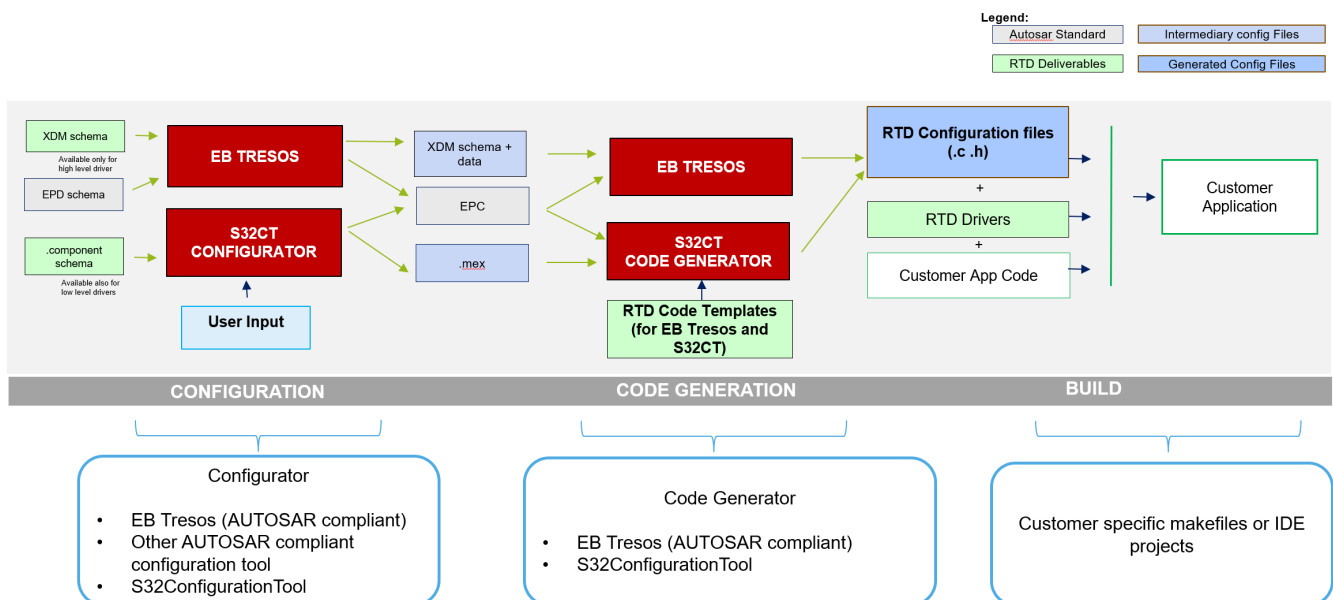


Figure 4.1 Configuration generation flow

- Ae
- AEC
- HVM

4.1 Ae

- Module [Ae](#)
 - Container [AeGeneral](#)
 - * Parameter [AeVersionInfoApi](#)
 - * Parameter [AeDevErrorDetect](#)
 - * Parameter [AeEnableUserModeSupport](#)
 - * Parameter [AeMultipartitionSupport](#)
 - * Reference [AeEcucPartitionRef](#)
 - * Container [AeGeneralVendorSpecific](#)
 - Parameter [AeTimeoutMethod](#)
 - Parameter [AeTimeoutDuration](#)
 - Container [AeHvmVmConfig](#)
 - * Parameter [AeHvmVoltageMonitorInputSelect](#)
 - * Parameter [AeVmHighVoltageDetectInterruptEn](#)
 - * Parameter [AeVmLowVoltageDetectInterruptEn](#)
 - * Parameter [AeHvmHighVoltageDetectEn](#)
 - * Parameter [AeHvmLowVoltageDetectEn](#)
 - * Parameter [AeHvmLowDetectRefVoltage](#)
 - * Parameter [AeHvmHighDetectRefVoltage](#)
 - Container [AeHvmHviConfig](#)
 - * Container [AeHvmHviGeneralConfig](#)
 - Parameter [AeHvmHviAnalogchannel](#)
 - Parameter [AeHvmAnalogInput](#)
 - Parameter [AeHvmUnderControlIsrCallback](#)
 - * Container [AeHvmHviInstanceConfig](#)
 - Parameter [AeHvmHviChannelIdx](#)
 - Parameter [AeHVIIInputDigitalEn](#)
 - Parameter [AeHVIIInterruptInputDigitalEn](#)
 - Parameter [AeHviDigitalInputPolarity](#)
 - Parameter [AeHviDividerSelect](#)
 - Parameter [AeHVIPullUpEn](#)
 - Parameter [AeHVIPullDownEn](#)
 - Container [AeSpi](#)
 - * Container [AeSpiSequence](#)
 - Parameter [AeSpiAccessSynchronous](#)
 - Reference [AeSpiSequenceName](#)

- Container [AeIrqConfig](#)
 - * Parameter [ClearEventsFaultsFlagEn](#)
 - * Container [AeIntConfig](#)
 - Parameter [AeIntName](#)
 - Parameter [AeIsrEnabled](#)
 - Parameter [AeIsrPriority](#)
 - Parameter [AeIntHandler](#)
- Container [AeGHSCConfig](#)
 - * Parameter [AeGHSProcessingCallout](#)
 - * Parameter [AeGHSHandlerEnable](#)
 - * Container [AeGHSHandlerConfig](#)
 - Parameter [AeGHSName](#)
 - Parameter [AeHandlerEnabled](#)
 - Parameter [AeGHSPriority](#)
 - Parameter [AeGHSHandler](#)
- Container [AeNMICConfig](#)
 - * Parameter [NMIInterruptEnable](#)
 - * Reference [AePlatformSourceName](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ModuleId](#)
 - * Parameter [VendorId](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)

4.1.1 Module Ae

Configuration of the Ae (Application Extension Driver) module.

Included containers:

- [AeGeneral](#)
- [AeHvmVmConfig](#)
- [AeHvmHviConfig](#)
- [AeSpi](#)
- [AeIrqConfig](#)

- [AeGHSCfg](#)
- [AeNMICfg](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-PRE-COMPILE, VARIANT-POST-BUILD

4.1.2 Container AeGeneral

General configuration of Ae Driver module.

Included subcontainers:

- [AeGeneralVendorSpecific](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.3 Parameter AeVersionInfoApi

Enables / Disables Ae_GetVersionInfo API.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.1.4 Parameter AeDevErrorDetect

Enables / Disables development error detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	AUTOSAR_ECUC
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.1.5 Parameter AeEnableUserModeSupport

When this parameter is enabled, the Ae module will adapt to run from User Mode.

For more information, please see chapter User Mode Support in IM

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.1.6 Parameter AeMultipartitionSupport

AeMultipartitionSupport

This parameter globally enables the possibility to support multi-partition. If this parameter is enabled, at least one EcucPartition needs to be defined (in all variants).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.7 Reference AeEcucPartitionRef

Maps the Ae driver to zero or multiple ECUC partitions to make the driver

API available in the according partition.

Property	Value
type	ECUC-REFERENCE-DEF
origin	AUTOSAR_ECUC
lowerMultiplicity	0
upperMultiplicity	Infinite
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.1.8 Container AeGeneralVendorSpecific

General vendor-specific configuration of Ae Driver.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.9 Parameter AeTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from Oslf will be used in the driver.

Configuration Parameters

Note: If SystemTimer or CustomTimer are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM', 'OSIF_COUNTER_DUMMY']

4.1.10 Parameter AeTimeoutDuration

The unit of measurement is given in number of microseconds.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	1000
max	65535
min	1

4.1.11 Container AeHvmVmConfig

General Configuration Volatage Monitor.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.12 Parameter AeHvmVoltageMonitorInputSelect

Select the voltage that is observed by the voltage monitor.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	VM_HIGH_VOLTAGE_INPUT
literals	['VM_HIGH_VOLTAGE_INPUT', 'VM_SENSE_VOLTAGE']

4.1.13 Parameter AeVmHighVoltageDetectInterruptEn

This parameter is used to enable/disable the digital input interrupt of High Voltage Detect.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.14 Parameter AeVmLowVoltageDetectInterruptEn

This parameter is used to enable/disable the digital input interrupt of Low Voltage Detect.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.15 Parameter AeHvmHighVoltageDetectEn

Enables / Disables the voltage monitor for high voltage detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.16 Parameter AeHvmLowVoltageDetectEn

Enables / Disables the voltage monitor for Low voltage detection.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.17 Parameter AeHvmLowDetectRefVoltage

Configures the reference voltage level for low voltage detection.

This reference voltage level is in datasheet.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	VM_LBI1
literals	['VM_LBI1', 'VM_LBI2', 'VM_LBI3', 'VM_LBI4']

4.1.18 Parameter AeHvmHighDetectRefVoltage

Configures the reference voltage level for high voltage detection.

This reference voltage level is in datasheet.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	VM_HBI1

Property	Value
literals	['VM_HBI1', 'VM_HBI2']

4.1.19 Container AeHvmHviConfig

Configuration of High Voltage Input for each instances.

Included subcontainers:

- [AeHvmHviGeneralConfig](#)
- [AeHvmHviInstanceConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.20 Container AeHvmHviGeneralConfig

Generic configuration the High Voltage Input.

Note: Implementation Specific Parameter.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.21 Parameter AeHvmHviAnalogchannel

Configures the Analog channel.

Enable the analog input for one HVI.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	HVI0_CHANNEL
literals	['HVI0_CHANNEL', 'HVI1_CHANNEL']

4.1.22 Parameter AeHvmAnalogInput

Configures the Analog Input.

Select the analog input that is given out on the analog output of the instance that is selected.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
default Value	HVI_ANALOG_INPUT_BUFFERED
literals	['HVI_ANALOG_OUT_OHMIC', 'HVI_ANALOG_INPUT_BUFFERED', 'HVI_ANALOG_DIVIDE_DOWN_HIGH_INPUT_BUFFERED', 'HVI_ANALOG_INPUT_UNBUFFERED']

4.1.23 Parameter AeHvmUnderControlIsrCallback

Enables callback function was handled by user for Hvm_Ip_pfReportCallback, if is disable, driver doesn't do anything, callback function is empty.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false

Property	Value
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.24 Container AeHvmHviInstanceConfig

Container information to configure for each HVI instance.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	255
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.25 Parameter AeHvmHviChannelIdx

High Voltage Input channel Index

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	1
min	0

4.1.26 Parameter AeHVIInputDigitalEn

This parameter is used to enable/disable the digital input.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.27 Parameter AeHVIIInterruptInputDigitalEn

This parameter is used to enable/disable the digital input interrupt of HVI channel.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.28 Parameter AeHviDigitalInputPolarity

Configures the Digital Input Polarity.

Configure whether the high or the low state on the digital input triggers an event.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE

Configuration Parameters

Property	Value
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	HVI_DIGITAL_INPUT_POLARITY_PULLDOWN
literals	['HVI_DIGITAL_INPUT_POLARITY_PULLDOWN', 'HVI_DIGITAL_INPUT_POLARITY_PULLUP']

4.1.29 Parameter AeHviDividerSelect

Configures the ratio of the voltage division.

The pull down must be enabled to divide by the ratios selectable here. To set a divider ratio of 1, the pull down must be disabled.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE VARIANT-POST-BUILD: POST-BUILD
defaultValue	HVI_VOLTAGE_DIVISION_2
literals	['HVI_VOLTAGE_DIVISION_2', 'HVI_VOLTAGE_DIVISION_6', 'HVI_VOLTAGE_DIVISION_11', 'HVI_VOLTAGE_DIVISION_16']

4.1.30 Parameter AeHVIPullUpEn

This parameter is used to enable/disable the pull up on the input of each instance HVI.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.31 Parameter AeHVIPullDownEn

This parameter is used to enable/disable the pull down on the input of each instance HVI.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.32 Container AeSpi

Configuration of one Application Extension SPI access (if SPI is used).

Included subcontainers:

- [AeSpiSequence](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.33 Container AeSpiSequence

Container gives Ae driver information about one SPI sequence.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	255
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE

4.1.34 Parameter AeSpiAccessSynchronous

This parameter is used to define whether the access to the Spi sequence is synchronous or asynchronous.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.35 Reference AeSpiSequenceName

Reference to a Spi sequence configuration container.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Spi/SpiDriver/SpiSequence

4.1.36 Container AeIrqConfig

Generic configuration for the interrupts (events, faults).

Included subcontainers:

- [AeIntConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A

4.1.37 Parameter ClearEventsFaultsFlagEn

Enable/Disable clear Event/Fault flag.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.38 Container AeIntConfig

Vendor specific:

Configuration for interrupt requests.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	25
upperMultiplicity	25
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.39 Parameter AeIntName

Vendor specific:

Interrupt Name caused by faults or events.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1

Configuration Parameters

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	GDU_FAULT_PROT
literals	['GDU_FAULT_PROT', 'GDU_INT', 'DPGA_OC_NEG', 'DPGA_OC_P↵OS', 'PMC_VLS', 'RAW_FAILED', 'CHKSUM_ERR', 'OBSCHK_ERR', 'X↵FER_ERR', 'EVENT_EXPIRED', 'MCU_SUPPLY', 'HVD_AE_INTERN', 'ILL_TEST', 'ALIVE_WDG', 'LIN_INT', 'OCD_VDDE', 'TEMP_WDG_P↵MC', 'TEMP_WDG_PHY', 'CAN_INT', 'LVD_VDDC', 'WAKEUP', 'HVI_↵ACTIVE', 'HVI_AE_SUPPLY', 'STATERESET', 'FRAMEWIDTH']

4.1.40 Parameter AeIsrEnabled

Vendor specific: Enable to raise interrupt_out for events/faults which owned by AEC.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.41 Parameter AeIsrPriority

Set priority to handle for event or fault which caused interrupt.

Priority's value smaller that will be handled first.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	24
min	0

4.1.42 Parameter AeIntHandler

Function to be installed as the interrupt handler.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	undefined_handler

4.1.43 Container AeGHSConfig

Generic configuration for the Global Health Status Handler .

Included subcontainers:

- [AeGHSHandlerConfig](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.44 Parameter AeGHSProcessingCallout

Function pointer to callback function. This function will be called for function within GHS fault processing.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-FUNCTION-NAME-DEF

Configuration Parameters

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	NULL_PTR

4.1.45 Parameter AeGHSHandlerEnable

Enable/Disable Global Health Status Handler.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.1.46 Container AeGHSHandlerConfig

Vendor specific:

Configuration for interrupt requests.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	4
upperMultiplicity	4
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.47 Parameter AeGHSName

Vendor specific:

Interrupt Name caused by faults or events.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	CLOCK_MISS
literals	['CLOCK_MISS', 'WRONG_ANSWER', 'FSM_STATUS', 'FRAME_COUNT']

4.1.48 Parameter AeHandlerEnabled

Vendor specific: Enable to raise interrupt_out for events/faults which owned by AEC.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.1.49 Parameter AeGHSPriority

Set priority to handle for event or fault which caused interrupt.

Priority's value smaller that will be handled first.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true

Configuration Parameters

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	3
min	0

4.1.50 Parameter AeGSHHandler

Function to be installed as the interrupt handler.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	undefined_handler

4.1.51 Container AeNMIconfig

Generic configuration for the interrupts (events, faults).

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.1.52 Parameter NMIIInterruptEnable

Enable/Disable clear Event/Fault flag.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.1.53 Reference AePlatformSourceName

Reference to a Spi sequence configuration container.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Platform/IntCtrlConfig/PlatformIsrConfig

4.1.54 Container CommonPublishedInformation

Common container, aggregated by all modules. It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A

4.1.55 Parameter ModuleId

Module ID of this module from the BSW Module List.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	250
max	250
min	250

4.1.56 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

4.1.57 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU,

BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name.

In total, the implementation specific name is generated as follows:

[ModuleName]_[VendorId]_[VendorApiInfix][API name from SWS].

E.g. Assuming that the VendorId of the implementor is 123 and the implementer chose a

VendorApiInfix of 'v11r456' an API named Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity > 1.

It shall not be used for modules with upper multiplicity = 1.

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.1.58 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.1.59 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	7
max	7
min	7

4.1.60 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.61 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	3
max	3
min	3

4.1.62 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.1.63 Parameter SwPatchVersion

Patch version number of the vendor specific implementation of the module. The numbering is vendor specific.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.2 AEC

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Aec](#)
 - Container [AeGeneral](#)
 - * Parameter [AeDevErrorDetect](#)
 - * Parameter [AeEnableUserModeSupport](#)
 - * Container [AeGeneralVendorSpecific](#)
 - Parameter [AeTimeoutMethod](#)
 - Parameter [AeTimeoutDuration](#)
 - Container [AeSpi](#)
 - * Container [AeSpiSequence](#)
 - Parameter [AeSpiAccessSynchronous](#)
 - Reference [AeSpiSequenceName](#)
 - Container [AeIrqConfig](#)
 - * Parameter [ClearEventsFaultsFlagEn](#)
 - * Container [AeIntConfig](#)
 - Parameter [AeIntName](#)
 - Parameter [AeIsrEnabled](#)
 - Parameter [AeIsrPriority](#)
 - Parameter [AeIntHandler](#)
 - Container [AeGHSCConfig](#)
 - * Parameter [AeGHSProcessingCallout](#)
 - * Parameter [AeGHSHandlerEnable](#)
 - * Container [AeGHSHandlerConfig](#)
 - Parameter [AeGHSName](#)
 - Parameter [AeHandlerEnabled](#)
 - Parameter [AeGHSPriority](#)
 - Parameter [AeGHSHandler](#)
 - Container [AeNMICConfig](#)
 - * Parameter [NMIInterruptEnable](#)
 - * Reference [AePlatformSourceName](#)

4.2.1 Module Aec

Configuration of the Aec (Application Extension Driver) module.

Included containers:

- [AeGeneral](#)

- [AeSpi](#)
- [AeIrqConfig](#)
- [AeGHSCConfig](#)
- [AeNMICConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-PRE-COMPILE;VARIANT-POST-BUILD

4.3 HVM

The low level driver component can only be used with S32CT Configurator in S32 Design Studio.

- Module [Hvm](#)
 - Container [AeGeneral](#)
 - * Parameter [AeDevErrorDetect](#)
 - * Parameter [AeEnableUserModeSupport](#)
 - * Container [AeGeneralVendorSpecific](#)
 - Parameter [AeTimeoutMethod](#)
 - Parameter [AeTimeoutDuration](#)
 - Container [AeHvmVmConfig](#)
 - * Parameter [AeHvmVoltageMonitorInputSelect](#)
 - * Parameter [AeVmHighVoltageDetectInterruptEn](#)
 - * Parameter [AeVmLowVoltageDetectInterruptEn](#)
 - * Parameter [AeHvmHighVoltageDetectEn](#)
 - * Parameter [AeHvmLowVoltageDetectEn](#)
 - * Parameter [AeHvmLowDetectRefVoltage](#)
 - * Parameter [AeHvmHighDetectRefVoltage](#)
 - Container [AeHvmHviConfig](#)
 - * Container [AeHvmHviGeneralConfig](#)
 - Parameter [AeHvmHviAnalogchannel](#)
 - Parameter [AeHvmAnalogInput](#)
 - Parameter [AeHvmUnderControlIsrCallback](#)
 - * Container [AeHvmHviInstanceConfig](#)
 - Parameter [AeHvmHviChannelIdx](#)

Configuration Parameters

- Parameter [AeHVIInputDigitalEn](#)
- Parameter [AeHVIInterruptInputDigitalEn](#)
- Parameter [AeHviDigitalInputPolarity](#)
- Parameter [AeHviDividerSelect](#)
- Parameter [AeHVIPullUpEn](#)
- Parameter [AeHVIPullDownEn](#)

4.3.1 Module Hvm

Configuration of the Hvm (Application Extension Driver) module.

Included containers:

- [AeGeneral](#)
- [AeHvmVmConfig](#)
- [AeHvmHviConfig](#)

Property	Value
type	MODULE-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-PRE-COMPILE;VARIANT-POST-BUILD



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

AEC Driver	46
HVM Driver	52
Aeernet Driver	58

Chapter 6

Module Documentation

6.1 AEC Driver

6.1.1 Detailed Description

6.1.1.1 Functions Expectation called for read/write value from/to AE's registers in polling/interrupt mode:

- `Aec_Ip_Init`
- `Aec_Ip_SpiWrite`
- `Aec_Ip_SpiRead`

Data Structures

- struct `Aec_Ip_SpiConfigType`
SPI configuration for access. [More...](#)
- struct `Aec_Ip_IntRouteConfigType`
Structure storing the routing and handler configuration for an interrupt request. [More...](#)
- struct `Aec_Ip_GHSRouteConfigType`
Structure storing the routing and handler configuration for an interrupt request. [More...](#)

Types Reference

- typedef void(* `Aec_Ip_IntHandlerType`) (void)
Interrupt handler type.
- typedef void(* `Aec_Ip_GHSHandlerType`) (void)
Interrupt handler type.

Enum Reference

- enum `Aec_Ip_StatusType`
Status type.

Function Reference

- `Aec_Ip_StatusType Aec_Ip_Init` (const `Aec_Ip_ConfigType` *AecConfig)
Initializes the Aec Driver.

- [Aec_Ip_StatusType Aec_Ip_SpiWrite](#) (uint32 RegAddr, uint8 DataWidth, uint32 Data)
Write Value to indexed register by Register address through SPI.
- [Aec_Ip_StatusType Aec_Ip_SpiRead](#) (uint32 RegAddr, uint8 DataWidth, uint32 *Data)
Read Value return from indexed register by Register address through SPI.
- boolean [Aec_Ip_Spi_Frame_Counter](#) (boolean IsVerify, uint8 ExpectedFrameCount, uint8 *ActualFrameCount)
Verify for correctness the received frame counter or just read number of frames.
- uint16 [Aec_Ip_DecodeGlobalHealthStatus](#) (void)
Return status of global health.
- void [Aec_Ip_SetRaw](#) (boolean Activate)
Enable/Disable read after write.

6.1.2 Data Structure Documentation

6.1.2.1 struct Aec_Ip_SpiConfigType

SPI configuration for access.

Definition at line 149 of file Aec_Ip_Types.h.

6.1.2.2 struct Aec_Ip_IntRouteConfigType

Structure storing the routing and handler configuration for an interrupt request.

Definition at line 170 of file Aec_Ip_Types.h.

Data Fields

- uint16 [IsrMask](#)
Name of event/fault.
- uint8 [IsrPriority](#)
Priority for event/fault.
- [Aec_Ip_IntHandlerType pfHandler](#)
Interrupt handler.

Field Documentation

IsrMask

uint16 [IsrMask](#)

Name of event/fault.

Definition at line 173 of file Aec_Ip_Types.h.

IsrPriority

`uint8 IsrPriority`

Priority for event/fault.

Definition at line 175 of file `Aec_Ip_Types.h`.

pfHandler

`Aec_Ip_IntHandlerType pfHandler`

Interrupt handler.

Definition at line 177 of file `Aec_Ip_Types.h`.

6.1.2.3 struct Aec_Ip_GHSRouteConfigType

Structure storing the routing and handler configuration for an interrupt request.

Definition at line 184 of file `Aec_Ip_Types.h`.

Data Fields

<code>uint16</code>	Mask	Name of event/fault.
<code>uint8</code>	Priority	Priority for event/fault.
<code>Aec_Ip_GHSHandlerType</code>	pfHandler	Interrupt handler.

6.1.3 Types Reference

6.1.3.1 Aec_Ip_IntHandlerType

`typedef void(* Aec_Ip_IntHandlerType) (void)`

Interrupt handler type.

Definition at line 159 of file `Aec_Ip_Types.h`.

6.1.3.2 Aec_Ip_GHSHandlerType

`typedef void(* Aec_Ip_GHSHandlerType) (void)`

Interrupt handler type.

Definition at line 164 of file `Aec_Ip_Types.h`.

6.1.4 Enum Reference

6.1.4.1 Aec_Ip_StatusType

`enum Aec_Ip_StatusType`

Status type.

Enumerator

AEC_STATUS_SUCCESS	Generic operation success status
AEC_STATUS_ERROR	Generic operation failure status
AEC_STATUS_TIMEOUT	Generic operation timeout status

Definition at line 131 of file Aec_Ip_Types.h.

6.1.5 Function Reference

6.1.5.1 Aec_Ip_Init()

```
Aec_Ip_StatusType Aec_Ip_Init (
    const Aec_Ip_ConfigType * AecConfig )
```

Initializes the Aec Driver.

The configuration pointer is internally stored and the driver is initialized.

Returns

AEC_STATUS_SUCCESS if initialized successful. AEC_STATUS_TIMEOUT if initialized failed.

6.1.5.2 Aec_Ip_SpiWrite()

```
Aec_Ip_StatusType Aec_Ip_SpiWrite (
    uint32 RegAddr,
    uint8 DataWidth,
    uint32 Data )
```

Write Value to indexed register by Register address through SPI.

Parameters

in	<i>RegAddr</i>	Register address want to write to.
in	<i>DataWidth</i>	Size of register in AEC that want to read/write.
in	<i>Data</i>	Value want to write to indexed register.

Returns

Status of transmission

6.1.5.3 Aec_Ip_SpiRead()

```
Aec_Ip_StatusType Aec_Ip_SpiRead (
    uint32 RegAddr,
    uint8 DataWidth,
    uint32 * Data )
```

Read Value return from indexed register by Register address through SPI.

Parameters

in	<i>RegAddr</i>	Register address want to read from.
in	<i>DataWidth</i>	Size of register in AEC that want to read/write.
out	<i>Data</i>	Value want to read from indexed register.

Returns

Status of transmission

6.1.5.4 Aec_Ip_Spi_Frame_Counter()

```
boolean Aec_Ip_Spi_Frame_Counter (
    boolean IsVerify,
    uint8 ExpectedFrameCount,
    uint8 * ActualFrameCount )
```

Verify for correctness the received frame counter or just read number of frames.

Parameters

in	<i>IsVerify</i>	want to verify for correctness for frame counter.
in	<i>ExpectedFrameCount</i>	Number of frame expected get.
out	<i>ActualFrameCount</i>	Number of frame get in real.

Returns

TRUE if verify correct.

6.1.5.5 Aec_Ip_DecodeGlobalHealthStatus()

```
uint16 Aec_Ip_DecodeGlobalHealthStatus (
    void )
```

Return status of global health.

6.1.5.6 Aec_Ip_SetRaw()

```
void Aec_Ip_SetRaw (
    boolean Activate )
```

Enable/Disable read after write.

6.2 HVM Driver

6.2.1 Detailed Description

6.2.1.1 Functions Expectation called first for configuration initilized:

- `Hvm_Ip_Init`

Data Structures

- struct [Ae_HviHighVoltageInputConfig](#)
Ae_HviHighVoltageInputConfig_structure. [More...](#)
- struct [Ae_VmConfig](#)
Ae_VmConfig_structure. [More...](#)
- struct [Ae_HviChannelConfig](#)
Ae_HviChannelConfig. [More...](#)

Enum Reference

- enum [Hvm_Ip_StatusType](#)
Hvm_Ip_StatusType_Enumeration.
- enum [Hvm_Ip_OutputStatusType](#)
Hvm_Ip_OutputStatusType_Enumeration.
- enum [Hvm_Ip_ChannelType](#)
Hvm_Ip_ChannelType_Enumeration.
- enum [Hvm_Ip_HighVoltDetectRefType](#)
Hvm_Ip_HighVoltDetectRefType_Enumeration.
- enum [Hvm_Ip_LowVoltDetectRefType](#)
Hvm_Ip_LowVoltDetectRefType_Enumeration.
- enum [Hvm_Ip_VoltageMonitorInputType](#)
Hvm_Ip_VoltageMonitorInputType_Enumeration.
- enum [Hvm_Ip_DigitalInputPolarityType](#)
Hvm_Ip_DigitalInputPolarityType_Enumeration.
- enum [Ae_AnalogInputType](#)
Ae_AnalogInputType_Enumeration.
- enum [Hvm_Ip_DivVoltageType](#)
Hvm_Ip_DivVoltageType_Enumeration.

Function Reference

- [Hvm_Ip_StatusType Hvm_Ip_Init](#) (const [Hvm_Ip_ConfigType](#) *pHvmConfig)
Initializes the Hvm Driver.
- [Hvm_Ip_StatusType Hvm_Ip_AdcMeasureConfig](#) (const [Ae_HviHighVoltageInputConfig](#) *pHviHighVolt↵
Config, [Ae_AnalogInputType](#) AnalogInputSelect)
Initializes the configuration for analog input for measurement.
- [Hvm_Ip_StatusType Hvm_Ip_DigitalInputEventConfig](#) (const [Ae_HviChannelConfig](#) *pHviConfig)

Initializes the configuration for digital input.

- [Hvm_Ip_StatusType](#) [Hvm_Ip_VoltageMonitorConfig](#) (const [Ae_VmConfig](#) *pAeVmConfig, const [Ae_HviHighVoltageInputConfig](#) *pHighVoltageInput)

Initializes the configuration for Voltage Monitor.

- void [Hvm_Ip_ReadFaultsEventsStatus](#) (void)

Check status flag used in polling mode.

- void [Hvm_Ip_FaultsEventsProccess](#) (void)

Check and clear interrupt flag used in interrupt mode.

6.2.2 Data Structure Documentation

6.2.2.1 struct Ae_HviHighVoltageInputConfig

[Ae_HviHighVoltageInputConfig](#)_structure.

Definition at line 210 of file [Hvm_Ip_Types.h](#).

6.2.2.2 struct Ae_VmConfig

[Ae_VmConfig](#)_structure.

Definition at line 222 of file [Hvm_Ip_Types.h](#).

6.2.2.3 struct Ae_HviChannelConfig

[Ae_HviChannelConfig](#).

Definition at line 237 of file [Hvm_Ip_Types.h](#).

6.2.3 Enum Reference

6.2.3.1 Hvm_Ip_StatusType

enum [Hvm_Ip_StatusType](#)

[Hvm_Ip_StatusType](#)_Enumeration.

Enumerator

HVM_STATUS_SUCCESS	Generic operation success status
HVM_STATUS_ERROR	Generic operation failure status
HVM_STATUS_TIMEOUT	Generic operation timeout status

Definition at line 99 of file [Hvm_Ip_Types.h](#).

6.2.3.2 Hvm_Ip_OutputStatusType

enum [Hvm_Ip_OutputStatusType](#)

[Hvm_Ip_OutputStatusType](#)_Enumeration.

Enumerator

HVM_STATUS_NOTHING_DETECT	Generic operation nothing detect status
HVM_STATUS_HIGH_DETECT	Generic operation high voltage detect status
HVM_STATUS_LOW_DETECT	Generic operation low voltage detect status
HVM_STATUS_DIGITAL_INPUT_ACTIVE	Generic operation digital input active status
HVM_STATUS_OUTPUT_ERROR	Generic operation failure status

Definition at line 112 of file Hvm_Ip_Types.h.

6.2.3.3 Hvm_Ip_ChannelType

enum [Hvm_Ip_ChannelType](#)

Hvm_Ip_ChannelType_Enumeration.

Definition at line 127 of file Hvm_Ip_Types.h.

6.2.3.4 Hvm_Ip_HighVoltDetectRefType

enum [Hvm_Ip_HighVoltDetectRefType](#)

Hvm_Ip_HighVoltDetectRefType_Enumeration.

Definition at line 137 of file Hvm_Ip_Types.h.

6.2.3.5 Hvm_Ip_LowVoltDetectRefType

enum [Hvm_Ip_LowVoltDetectRefType](#)

Hvm_Ip_LowVoltDetectRefType_Enumeration.

Definition at line 147 of file Hvm_Ip_Types.h.

6.2.3.6 Hvm_Ip_VoltageMonitorInputType

enum [Hvm_Ip_VoltageMonitorInputType](#)

Hvm_Ip_VoltageMonitorInputType_Enumeration.

Definition at line 159 of file Hvm_Ip_Types.h.

6.2.3.7 Hvm_Ip_DigitalInputPolarityType

enum [Hvm_Ip_DigitalInputPolarityType](#)

Hvm_Ip_DigitalInputPolarityType_Enumeration.

Definition at line 169 of file Hvm_Ip_Types.h.

6.2.3.8 Ae_AnalogInputType

enum `Ae_AnalogInputType`

`Ae_AnalogInputType_Enumeration`.

Definition at line 179 of file `Hvm_Ip_Types.h`.

6.2.3.9 Hvm_Ip_DivVoltageType

enum `Hvm_Ip_DivVoltageType`

`Hvm_Ip_DivVoltageType_Enumeration`.

Definition at line 191 of file `Hvm_Ip_Types.h`.

6.2.4 Function Reference

6.2.4.1 Hvm_Ip_Init()

```
Hvm_Ip_StatusType Hvm_Ip_Init (
    const Hvm_Ip_ConfigType * pHvmConfig )
```

Initializes the Hvm Driver.

The configuration pointer is internally stored and the driver is initialized.

Parameters

in	<i>HvmConfig</i>	Pointer to the module configuration structure
----	------------------	---

Return values

<i>HVM_STATUS_SUCCESS</i>	The initialization was successful.
---------------------------	------------------------------------

6.2.4.2 Hvm_Ip_AdcMeasureConfig()

```
Hvm_Ip_StatusType Hvm_Ip_AdcMeasureConfig (
    const Ae_HviHighVoltageInputConfig * pHviHighVoltConfig,
    Ae_AnalogInputType AnalogInputSelect )
```

Initializes the configuration for analog input for measurement.

Base on configuration to select analog input, HVI channel, divider if high voltage input selected.

Parameters

in	<i>pHviHighVoltConfig</i>	Pointer to configuration for High Voltage Input. AnalogInputSelect Type of analog input.
----	---------------------------	--

Return values

<i>HVM_STATUS_SUCCESS</i>	The initialization was successful.
---------------------------	------------------------------------

6.2.4.3 Hvm_Ip_DigitalInputEventConfig()

```
Hvm_Ip_StatusType Hvm_Ip_DigitalInputEventConfig (
    const Ae_HviChannelConfig * pHviConfig )
```

Initializes the configuration for digital input.

Initializes the configuration for digital input include: voltage input source, divider, mode polling or interrupt used.

Parameters

in	<i>pHviConfig</i>	HVI channels configuration.
----	-------------------	-----------------------------

Return values

<i>HVM_STATUS_SUCCESS</i>	The initialization was successful.
---------------------------	------------------------------------

6.2.4.4 Hvm_Ip_VoltageMonitorConfig()

```
Hvm_Ip_StatusType Hvm_Ip_VoltageMonitorConfig (
    const Ae_VmConfig * pAeVmConfig,
    const Ae_HviHighVoltageInputConfig * pHighVoltageInput )
```

Initializes the configuration for Voltage Monitor.

Initializes the configuration for High/Low voltage detect.

Parameters

in	<i>pAeVmConfig</i>	Pointer to configuration of VM.
in	<i>pHighVoltageInput</i>	Pointer to configuration of High Voltage Input.

Return values

<i>HVM_STATUS_SUCCESS</i>	The initialization was successful.
---------------------------	------------------------------------

6.2.4.5 Hvm_Ip_ReadFaultsEventsStatus()

```
void Hvm_Ip_ReadFaultsEventsStatus (
    void )
```

Check status flag used in polling mode.

This function used to read status flag for digital input, high voltage, low voltage detect.

6.2.4.6 Hvm_Ip_FaultsEventsProccess()

```
void Hvm_Ip_FaultsEventsProccess (  
    void )
```

Check and clear interrupt flag used in interrupt mode.

Check interrupt flag, clear flag if event (digital input, high voltage, low voltage detected) occurred.

6.3 Aeernet Driver

6.3.1 Detailed Description

Data Structures

- struct [Ae_Ipw_ConfigType](#)

The structure contains the hardware controller configuration type. [More...](#)

Macros

- `#define AE_SID_INIT`
Service ID of Ae_Init.
- `#define AE_SID_WRITEREGISTER`
Service ID of Ae_WriteRegister.
- `#define AE_SID_READREGISTER`
Service ID of Ae_ReadRegister.
- `#define AE_SID_GETVERSIONINFO`
Service ID of Ae_GetVersionInfo.
- `#define AE_SID_ADCMEASURECONFIG`
Service ID of Ae_AdcMeasureConfig.
- `#define AE_SID_DIGITALINPUTCONFIG`
Service ID of Ae_DigitalInputEventConfig.
- `#define AE_SID_VOLTAGEMONITORCONFIG`
Service ID of Ae_VoltageMonitorConfig.
- `#define AE_SID_DECODEGLOBALHEALTHSTATUS`
Service ID of Ae_DecodeGlobalHealthStatus.
- `#define AE_SID_FRAMECOUNTER`
Service ID of Ae_Frame_Counter.
- `#define AE_SID_FAULTSEVENTSPROCESS`
Service ID of Ae_FaultsEventsProcess.
- `#define AE_SID_READFAULTSEVENTSSTATUS`
Service ID of Ae_ReadFaultsEventsStatus.
- `#define AE_SID_SETRAW`
Service ID of Ae_SetRaw.
- `#define AE_E_INV_CTRL_IDX`
Development Error ID for "Invalid Controller Index".
- `#define AE_E_UNINIT`
Development Error ID for "Uninitialized Controller".
- `#define AE_E_PARAM_POINTER`
Development Error ID for "Invalid (Null) Pointer Parameter".
- `#define AE_E_INV_PARAM`
Development Error ID for "Invalid Parameter".
- `#define AE_E_INV_MODE`

Development Error ID for "Invalid Controller Mode".

- #define [AE_E_INIT_FAILED](#)

Development Error ID for "Invalid Configuration pointer argument".

- #define [AE_E_ALREADY_INITIALIZED](#)

Development Error ID for "API Ae_Init service called while the Ae driver has already been initialized.".

- #define [AE_E_PARAM_CONFIG](#)

Development Error ID for "API service called with wrong resource assigned.".

Enum Reference

- enum [Ae_StateType](#)

State init.

Function Reference

- void [Ae_Init](#) (const [Ae_ConfigType](#) *CfgPtr)

Initializes the Ae Driver.

- Std_ReturnType [Ae_ReadRegister](#) (uint32 RegAddr, uint8 DataWidth, uint32 *RegValPtr)

Read register value.

- Std_ReturnType [Ae_WriteRegister](#) (uint32 RegAddr, uint8 DataWidth, uint32 RegVal)

Write register value.

- uint16 [Ae_DecodeGlobalHealthStatus](#) (void)

Return status of global health.

- boolean [Ae_Frame_Counter](#) (boolean IsVerify, uint8 ExpectedFrameCount, uint8 *ActualFrameCount)

Verify for correctness the received frame counter or just number of frames.

- Std_ReturnType [Ae_AdcMeasureConfig](#) (const [Ae_HviHighVoltageInputConfig](#) *pHviHighVoltConfig, [Ae_AnalogInputType](#) AnalogInputSelect)

Initializes the configuration for analog input for measurement.

- Std_ReturnType [Ae_DigitalInputEventConfig](#) (const [Ae_HviChannelConfig](#) *pAeHviChannelConfig)

Initializes the configuration for digital input.

- Std_ReturnType [Ae_VoltageMonitorConfig](#) (const [Ae_VmConfig](#) *pAeVmConfig, const [Ae_HviHighVoltageInputConfig](#) *pHighVoltageInput)

Initializes the configuration for High/Low Voltage detect.

- void [Ae_FaultsEventsProcess](#) (void)

Check and clear interrupt flag used in interrupt mode.

- void [Ae_ReadFaultsEventsStatus](#) (void)

Check status flag used in polling mode for digital input, high/low voltage detect.

- void [Ae_SetRaw](#) (boolean Activate)

Enable/Disable read after write.

- void [Ae_GetVersionInfo](#) (Std_VersionInfoType *VersionInfoPtr)

Returns the version information of this module.

6.3.2 Data Structure Documentation

6.3.2.1 struct Ae_Ipw_ConfigType

The structure contains the hardware controller configuration type.

Definition at line 110 of file CDD_Ae_Ipw_Types.h.

6.3.3 Macro Definition Documentation

6.3.3.1 AE_SID_INIT

```
#define AE_SID_INIT
```

Service ID of Ae_Init.

Definition at line 80 of file CDD_Ae.h.

6.3.3.2 AE_SID_WRITEREGISTER

```
#define AE_SID_WRITEREGISTER
```

Service ID of Ae_WriteRegister.

Definition at line 82 of file CDD_Ae.h.

6.3.3.3 AE_SID_READREGISTER

```
#define AE_SID_READREGISTER
```

Service ID of Ae_ReadRegister.

Definition at line 84 of file CDD_Ae.h.

6.3.3.4 AE_SID_GETVERSIONINFO

```
#define AE_SID_GETVERSIONINFO
```

Service ID of Ae_GetVersionInfo.

Definition at line 86 of file CDD_Ae.h.

6.3.3.5 AE_SID_ADCMEASURECONFIG

```
#define AE_SID_ADCMEASURECONFIG
```

Service ID of Ae_AdcMeasureConfig.

Definition at line 88 of file CDD_Ae.h.

6.3.3.6 AE_SID_DIGITALINPUTCONFIG

```
#define AE_SID_DIGITALINPUTCONFIG
```

Service ID of Ae_DigitalInputEventConfig.

Definition at line 90 of file CDD_Ae.h.

6.3.3.7 AE_SID_VOLTAGEMONITORCONFIG

```
#define AE_SID_VOLTAGEMONITORCONFIG
```

Service ID of Ae_VoltageMonitorConfig.

Definition at line 92 of file CDD_Ae.h.

6.3.3.8 AE_SID_DECODEGLOBALHEALTHSTATUS

```
#define AE_SID_DECODEGLOBALHEALTHSTATUS
```

Service ID of Ae_DecodeGlobalHealthStatus.

Definition at line 94 of file CDD_Ae.h.

6.3.3.9 AE_SID_FRAMECOUNTER

```
#define AE_SID_FRAMECOUNTER
```

Service ID of Ae_Frame_Counter.

Definition at line 96 of file CDD_Ae.h.

6.3.3.10 AE_SID_FAULTSEVENTSPROCESS

```
#define AE_SID_FAULTSEVENTSPROCESS
```

Service ID of Ae_FaultsEventsProcess.

Definition at line 98 of file CDD_Ae.h.

6.3.3.11 AE_SID_READFAULTSEVENTSSTATUS

```
#define AE_SID_READFAULTSEVENTSSTATUS
```

Service ID of Ae_ReadFaultsEventsStatus.

Definition at line 100 of file CDD_Ae.h.

6.3.3.12 AE_SID_SETRAW

```
#define AE_SID_SETRAW
```

Service ID of Ae_SetRaw.

Definition at line 102 of file CDD_Ae.h.

6.3.3.13 AE_E_INV_CTRL_IDX

```
#define AE_E_INV_CTRL_IDX
```

Development Error ID for "Invalid Controller Index".

Definition at line 108 of file CDD_Ae.h.

6.3.3.14 AE_E_UNINIT

```
#define AE_E_UNINIT
```

Development Error ID for "Uninitialized Controller".

Definition at line 110 of file CDD_Ae.h.

6.3.3.15 AE_E_PARAM_POINTER

```
#define AE_E_PARAM_POINTER
```

Development Error ID for "Invalid (Null) Pointer Parameter".

Definition at line 112 of file CDD_Ae.h.

6.3.3.16 AE_E_INV_PARAM

```
#define AE_E_INV_PARAM
```

Development Error ID for "Invalid Parameter".

Definition at line 114 of file CDD_Ae.h.

6.3.3.17 AE_E_INV_MODE

```
#define AE_E_INV_MODE
```

Development Error ID for "Invalid Controller Mode".

Definition at line 116 of file CDD_Ae.h.

6.3.3.18 AE_E_INIT_FAILED

```
#define AE_E_INIT_FAILED
```

Development Error ID for "Invalid Configuration pointer argument".

Definition at line 118 of file CDD_Ae.h.

6.3.3.19 AE_E_ALREADY_INITIALIZED

```
#define AE_E_ALREADY_INITIALIZED
```

Development Error ID for "API Ae_Init service called while the Ae driver has already been initialized".

Definition at line 120 of file CDD_Ae.h.

6.3.3.20 AE_E_PARAM_CONFIG

```
#define AE_E_PARAM_CONFIG
```

Development Error ID for "API service called with wrong resource assigned".

Definition at line 122 of file CDD_Ae.h.

6.3.4 Enum Reference**6.3.4.1 Ae_StateType**

```
enum Ae_StateType
```

State init.

Definition at line 153 of file CDD_Ae.h.

6.3.5 Function Reference**6.3.5.1 Ae_Init()**

```
void Ae_Init (
    const Ae_ConfigType * CfgPtr )
```

Initializes the Ae Driver.

The configuration pointer is internally stored and the driver is initialized.

6.3.5.2 Ae_ReadRegister()

```
Std_ReturnType Ae_ReadRegister (
    uint32 RegAddr,
    uint8 DataWidth,
    uint32 * RegValPtr )
```

Read register value.

Read register value.

Parameters

in	<i>RegAddr</i>	Index of the AE register to be read.
in	<i>DataWidth</i>	Size of register in AEC that want to read/write.
out	<i>RegValPtr</i>	Filled with the register content of the indexed register

Return E_OK if read value from register successful. E_NOT_OK E_OK if read value from register failure.

6.3.5.3 Ae_WriteRegister()

```
Std_ReturnType Ae_WriteRegister (
    uint32 RegAddr,
    uint8 DataWidth,
    uint32 RegVal )
```

Write register value.

Write register value.

Parameters

in	<i>RegAddr</i>	Index of the AE register to be write.
in	<i>DataWidth</i>	Size of register in AEC that want to read/write.
in	<i>RegVal</i>	Filled with the register content of the indexed register

Return E_OK if write value from register successful. E_NOT_OK E_OK if write value from register failure.

6.3.5.4 Ae_DecomposeGlobalHealthStatus()

```
uint16 Ae_DecomposeGlobalHealthStatus (
    void )
```

Return status of global health.

6.3.5.5 Ae_Frame_Counter()

```
boolean Ae_Frame_Counter (
    boolean IsVerify,
    uint8 ExpectedFrameCount,
    uint8 * ActualFrameCount )
```

Verify for correctness the received frame counter or just number of frames.

Parameters

in	<i>IsVerify</i>	want to verify for correctness for frame counter.
in	<i>ExpectedFrameCount</i>	Number of frame expected get.
out	<i>ActualFrameCount</i>	Number of frame get in real.

Return counter status of respond frame from AE_Robin.

6.3.5.6 Ae_AdcMeasureConfig()

```
Std_ReturnType Ae_AdcMeasureConfig (
    const Ae_HviHighVoltageInputConfig * pHviHighVoltConfig,
    Ae_AnalogInputType AnalogInputSelect )
```

Initializes the configuration for analog input for measurement.

Parameters

in	<i>pHviHighVoltConfig</i>	Pointer for configuration of High Voltage Input.
in	<i>AnalogInputSelect</i>	Type of Analog Input.

Return E_OK The initialization was successful.

6.3.5.7 Ae_DigitalInputEventConfig()

```
Std_ReturnType Ae_DigitalInputEventConfig (
    const Ae_HviChannelConfig * pAeHviChannelConfig )
```

Initializes the configuration for digital input.

Parameters

in	<i>pAeHviChannelConfig</i>	Pointer to configuration of HVI channel.
----	----------------------------	--

Return E_OK The initialization was successful.

6.3.5.8 Ae_VoltageMonitorConfig()

```
Std_ReturnType Ae_VoltageMonitorConfig (
    const Ae_VmConfig * pAeVmConfig,
    const Ae_HviHighVoltageInputConfig * pHighVoltageInput )
```

Initializes the configuration for High/Low Voltage detect.

Parameters

in	<i>pAeVmConfig</i>	Pointer to configuration of VM.
in	<i>pHighVoltageInput</i>	Pointer to configuration of High Voltage Input.

Return E_OK The initialization was successful.

6.3.5.9 Ae_FaultsEventsProcess()

```
void Ae_FaultsEventsProcess (
    void )
```

Check and clear interrupt flag used in interrupt mode.

Return Interrupt flag want to read raised or not.

6.3.5.10 Ae_ReadFaultsEventsStatus()

```
void Ae_ReadFaultsEventsStatus (
```



Module Documentation

```
void )
```

Check status flag used in polling mode for digital input, high/low voltage detect.

6.3.5.11 Ae_SetRaw()

```
void Ae_SetRaw (
    boolean Activate )
```

Enable/Disable read after write.

6.3.5.12 Ae_GetVersionInfo()

```
void Ae_GetVersionInfo (
    Std_VersionInfoType * VersionInfoPtr )
```

Returns the version information of this module.

Parameters

out	<i>VersionInfoPtr</i>	Pointer where to store the version information of this particular module instance. implements Ae_GetVersionInfo_Activity
-----	-----------------------	---

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Limited warranty and liability - Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes - NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use in automotive applications - This NXP product has been qualified for use in automotive applications. If this product is used by customer in the development of, or for incorporation into, products or services (a) used in safety critical applications or (b) in which failure could lead to death, personal injury, or severe physical or environmental damage (such products and services hereinafter referred to as "Critical Applications"), then customer makes the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, safety, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP. As such, customer assumes all risk related to use of any products in Critical Applications and NXP and its suppliers shall not be liable for any such use by customer. Accordingly, customer will indemnify and hold NXP harmless from any claims, liabilities, damages and associated costs and expenses (including attorneys' fees) that NXP may incur related to customer's incorporation of any product in a Critical Application.

Applications - Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale - NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at <http://www.nxp.com/profile/terms>, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control - This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at <mailto:PSIRT@nxp.com>) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. - NXP B.V. is not an operating company and it does not distribute or sell products.